

There and Back Again

Graphics Education with a View Towards the Future

Edward Angel
University of New Mexico
Santa Fe, NM, USA
angel@cs.unm.edu

Dave Shreiner
Unity Technologies
San Francisco, CA, USA
shreiner@siggraph.org

ABSTRACT

Teaching computer graphics has always been a juggling act between teaching fundamental techniques and expressing those techniques through graphics APIs. This talk surveys different teaching options over time, including our techniques expressed in our widely-used textbook. We consider the relevance of teaching graphics APIs given their evolution, and present a proposal for evolving teaching graphics.

ACM Reference Format:

Edward Angel and Dave Shreiner. 2022. There and Back Again: Graphics Education with a View Towards the Future. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH '22 Educator's Forum)*, August 07-11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532724.3535594>

1 INTRODUCTION

Computer graphics is a field that exploded in techniques and approaches over its at least six-decade lifetime. It's been a standard course incorporated into most computer science and engineering programs for at least the last 25 years. When computer graphics courses were first widely taught during the 1980s, there was limited hardware available to most students, coupled with a lack of standard graphics APIs for providing a common way to express the necessary techniques. Consequently, the courses taught in that era tended to be bottom-up, focusing on algorithms, going from pixels to lines to polygons and their displays. Most courses lacked the essential programming component.

From the simplest renderings on vector displays to the photorealistic renderings of today, the field has relied on numerous methods to expand its practitioners' capabilities. At the undergraduate level, there are numerous challenges to getting that learning underway: what programming language should be used; which hardware (if any) should be leveraged, and how much of its capabilities; and what techniques add a practical element to the student's education?

From the early educational texts [Foley and van Dam 1982; Hearn and Baker 1986], teaching computer graphics leveraged libraries to help convey the techniques. Early graphics specifications, such as Foley and van Dam's *SGP*, were very platform specific and limited the opportunities to those with access to the appropriate computing

setup. This situation was drastically improved with the introduction of cross-platform graphics APIs, most notably OpenGL, which became a widely used approach for teaching introductory graphics. It was at that point it became clear that we could develop an undergraduate-level computer graphics course that would be simple to present, work in most circumstances and not be limited by hardware availability, and would deliver tangible skills for those students entering industry after graduation. OpenGL, and now WebGL to some extent, have remained the standard APIs for teaching a first course in computer graphics. Our textbook, which is in its eighth edition, adopted WebGL two editions ago, with that change being positively received.

2 ENHANCED SIMPLICITY

Early versions of OpenGL implemented a *fixed-function graphics pipeline* that while extensive, exposed a limited set of graphics capabilities through a procedural interface. This version, while constrained, enabled rich discussions of elementary graphics techniques and allowed almost instant success for most students.

With the introduction of *shaders* to the pipeline, the ability to implement a broader set of techniques became available, but for the educator, at the expense of needing to first teach the fundamental skills of working with shaders. The complexity of introducing yet another programming language (e.g., the domain-specific *OpenGL Shading Language* aka *GLSL*), their compilation requirements, and the dataflow model they impose, added overhead that tended to confuse students and delayed learning the more relevant course content. Added to that was the dependence on the particular operating- and windowing-system, compiler toolchain, and system-dependent libraries, which also increased the time to get to a good learning environment.

With the introduction of WebGL [The WebGL Working Group 2011] we found an environment that abstracted system dependencies like opening application windows or processing input events. We pivoted our own book [Angel and Shreiner 2020] to a common environment using HTML5, JavaScript, and WebGL. The ability to open a window with a single line of HTML and skip dealing with compilers, linkers, and libraries was well received by educators and students. While WebGL still had the complexities of the dataflow model of shaders, and some quirkiness in types, it was much easier to achieve success with students.

3 EVOLUTION

However, within the gaming and graphics hardware industries, *immediate-mode*, state-machine based, procedural graphics interfaces were losing favor. The inability to know when graphics commands were going to be executed by the hardware, the amounts of

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Educator's Forum, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9366-9/22/08.

<https://doi.org/10.1145/3532724.3535594>

available memory, access to the various compute resources, and a myriad of other concerns led to the creation of a new generation of graphics APIs. Apple Computer's Metal [Apple 2022], Khronos' Vulkan [The Vulkan Working Group 2022], and Microsoft's Direct3D12 [Microsoft 2021]¹ are a new breed of graphics APIs that expose a much broader set of capabilities requiring a considerable set of knowledge to even be able to create the simplest applications (to get to the first triangle, a Vulkan application is about 1200 lines of code not including windowing or shaders). While it is possible to create courseware around such APIs, it's hardly a practical choice for either the student or the instructor.

Additionally, scene graph APIs such as *Three.js*, provide a higher level interface built on top of WebGL. Consequently, applications can be written using high-level functions in the API but without explicitly writing shaders and still achieve comparative speeds to pure WebGL applications. Such APIs present a much more of a top-down approach but one that requires only minor changes to the "standard" curriculum. The situation is similar with AR/VR and game software.

Further, an entirely different class of APIs support totally non-visual areas but rely on the same graphics pipelines to display. One example is Matlab that is used by millions of engineers, mathematicians, and scientists to do complex matrix calculations and simulations, the results of which are displayed with a variety of two- and three-dimensional plots and graphs. Underneath the hood, there is the graphics pipeline that uses the local GPU through means like OpenGL or WebGL. Consequently, an animated, shaded, hidden-surface-removed display is produced with a single function and executes at almost the same speed as separate OpenGL/WebGL program.

Conversely, the need to be able to program using a common graphics API seems to be diminishing, replaced with software tools built to generate applications and simplify development. With readily available software (particularly for academic institutions) like general real-time rendering systems like Unreal Engine or Unity; open-source software layers like Three.js, OpenSceneGraph, and Cocos2D; or general computation engines like MATLAB or Mathematica, all of which abstract – to differing degrees – the intricacies of dealing with a machine-level graphics API, we think unused approaches to teaching graphics techniques may be available for educators.

4 CONCLUSION

We've experimented with expressing many of the example problems used in our textbook in these new modalities. The supplemental materials contains example code for generating a 2D Sierpinski gasket using WebGL, Three.js and MATLAB. The gasket has been one of the first examples in all eight editions of our textbook, and serves to introduce students to an API and to show the connection between computer graphics and many of the fundamentals students have studied in beginning programming and data structure classes. The code shows that the key points of this example are the same regardless of the API.

Our contention is that a future version of a standard course could focus more on the core topics that have characterized computer graphics courses, such as modeling, viewing, approaches to rendering, texture mapping and its myriad applications, and less on a particular API. There could still be a major programming component but each student could choose which API to use for their assignments and projects. Students today are far more sophisticated in their ability to move among languages and APIs. Such a course would allow students to continue on various paths forward armed with the core concepts of computer graphics.

REFERENCES

- Edward Angel and Dave Shreiner. 2020. *Interactive Computer Graphics: A Top-Down Approach using WebGL* (eighth ed.). Pearson, New York, NY.
- Apple. 2022. *Metal Overview*. Apple Computer Corporation. <https://developer.apple.com/metal/>
- James D. Foley and Andries van Dam. 1982. *Introduction to Computer Graphics* (first ed.). Addison-Wesley, Boston, MA.
- GPU for the Web Working Group. 2022. *WebGPU*. Technical Report. <https://www.w3.org/TR/webgpu/>
- Donald Hearn and Pauline M. Baker. 1986. *Computer Graphics* (first ed.). Prentice Hall, Hoboken, NJ.
- Microsoft. 2021. *Direct3D 12 graphics*. Microsoft Corporation. <https://docs.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics>
- The Vulkan Working Group. 2022. *Vulkan Developer Website*. The Khronos Group. <https://www.vulkan.org>
- The WebGL Working Group. 2011. *The WebGL 2.0 Specification*. The Khronos Group. <https://www.khronos.org/registry/webgl/specs/latest/2.0/>

¹W3C's WebGPU [GPU for the Web Working Group 2022] is not included in this list as it is not officially released at the time of this writing.